TECHNICAL PROJECT REPORT

Multimodal RAG
AI Knowledge Platform

Agentic retrieval - hybrid search + reranking - OCR & Gemini Vision

Async ingestion - PostgreSQL / Redis / Qdrant - full observability

Docker / CI-CD / Kubernetes deployment path

Version 2.0.0 | September 2026

TECHNOLOGY SUMMARY

Backend	Python 3.12 - FastAPI - SQLAlchemy 2.0 (async) - Alembic
AI / RAG	Gemini 2.5 Flash - BGE-small ONNX embeddings - Qdrant - BM25 + RRF
Frontend	React 19 - TypeScript - Vite - Tailwind CSS 4 - shadcn/ui
Data layer	PostgreSQL (SQLite dev) - Redis (in-memory fallback) - Qdrant
Quality	pytest (66 tests) - ruff - RAG evaluation & benchmark scripts
Delivery	Docker multi-stage - docker compose - GitHub Actions - Kubernetes manif

Evidence-based analysis of the repository Multimodal-AI-Chatbot | MIT License

Table of Contents

1 Executive Summary . . . 5

2 Project Overview . . . 5

2.1 Purpose and problem statement . . . 5

2.2 Objectives . . . 5

2.3 Scope, target users and use cases . . . 5

3 Functional Requirements . . . 6

4 Non-Functional Requirements . . . 7

5 Features & Functionalities . . . 7

6 Technology Stack . . . 8

7 System Architecture . . . 9

8 Detailed Component Architecture . . . 10

9 Frontend Architecture . . . 10

10 Backend Architecture . . . 11

10.1 Middleware and error handling . . . 11

11 API Documentation . . . 11

12 Database Architecture . . . 12

13 Authentication & Authorization . . . 13

14 AI / ML / LLM / RAG . . . 14

14.1 LLM provider . . . 14

14.2 Embeddings and vector store . . . 14

14.3 Chunking and ingestion . . . 14

14.4 Retrieval pipeline . . . 14

14.5 Agentic behavior, context and guardrails . . . 15

14.6 Evaluation and limitations . . . 15

15 Complete Application Workflows . . . 15

15.1 Chat request (happy path) . . . 15

15.2 Document ingestion . . . 15

15.3 Image analysis . . . 15

16 Complete Data Flow . . . 16

17 Project Folder Structure . . . 16

18 Module & Implementation Details . . . 17

18.1 services/chat_service.py — the orchestrator . . . 17

18.2 retrieval/hybrid.py — retrieval strategies . . . 17

18.3 llm/gemini_client.py — provider hardening . . . 17

18.4 ingestion/ — service, state machine, pipeline . . . 17

18.5 security/ modules . . . 17
18.6 frontend hooks . . . 17
19 Validation & Error Handling . . . 18
20 Security Analysis . . . 18
20.1 Implemented security . . . 18
20.2 Recommended security improvements . . . 18
21 Testing . . . 19
21.1 RAG evaluation (measured) . . . 19
25 Deployment & DevOps . . . 19
26 Installation & Setup . . . 20
26.1 Zero-config local development . . . 20
26.2 Full stack via Docker . . . 20
26.3 Tests, evaluation, production build . . . 20
27 Problems Found & Improvements Made . . . 20
27.1 Potential issues identified during analysis (open) . . . 21
28 Current Strengths . . . 21
29 Known Limitations . . . 21
30 Recommended Improvements . . . 22
30.1 High priority . . . 22
30.2 Medium priority . . . 22
30.3 Low priority . . . 22
31 Future Enhancements . . . 22
32 Maintenance & Support . . . 22
33 GitHub / Repository Readiness . . . 23
34 Resume / Portfolio Value . . . 23
35 Final Technical Summary . . . 24
36 Appendix . . . 24
36.1 Environment variables (names only — values are operator-provided) . . . 24
36.2 Glossary . . . 24
36.3 Verification environment . . . 25

1 Executive Summary

The **Multimodal RAG AI Knowledge Platform** is a full-stack, production-oriented retrieval-augmented generation system. Users ask questions or upload documents and images through a React SPA; the backend classifies each query, retrieves evidence from a hybrid dense+keyword index, grounds the answer with citations, and streams the response token by token while attributing token usage and estimated cost.

It solves a concrete enterprise problem: generic LLM chatbots hallucinate and cannot answer questions about private product, FAQ or documentation corpora. This platform combines document intelligence (OCR + Gemini Vision), an async ingestion pipeline with SHA-256 deduplication and versioning, tenant-aware retrieval, and a full observability and evaluation layer, so answers are traceable to source documents.

Target users include knowledge workers querying a private corpus (support, internal docs), platform engineers operating the deployment, and developers extending the RAG pipeline. All major subsystems are implemented and validated locally: **66 offline tests pass**, the API boots and serves /health, /ready and /metrics, retrieval evaluation reports a **0.93 hit-rate with MRR 0.92–1.0**, and the frontend builds successfully. Distributed features (multi-replica Redis, Qdrant cluster, Kubernetes deployment) are architecture-ready but were not deployed during this analysis — stated explicitly wherever relevant.

Verification performed during this analysis	Result	Status
Backend test suite (pytest tests -q)	66 passed, 0 failed	Pass
API startup + /api/v1/health	{"status":"ok"}	Pass
Readiness probe /api/v1/ready	postgresql: true, qdrant: true	Pass
Prometheus exposition /api/v1/metrics	HTTP 200 (~5.5 KB payload)	Pass
Frontend build (tsc -b && vite build)	Built in ~1.1 s; chunk-size advisory warning	Pass
Secrets scan of tracked files	No hardcoded credentials found; .env not committed	Pass

2 Project Overview

2.1 Purpose and problem statement

Enterprises hold knowledge in heterogeneous artifacts: JSON knowledge bases, PDFs, scanned images, FAQs and product sheets. Off-the-shelf assistants cannot access this private content and answer unreliably. The project provides a self-hosted assistant that ingests such artifacts asynchronously, indexes them into a vector database plus a keyword index, and answers questions with verifiable citations, bounded cost and groundedness checks.

2.2 Objectives

- **Grounded answers** — knowledge answers cite retrieved sources with titles, pages and scores.
- **Production readiness** — rate limiting, retries, dead-letter queues, health probes, metrics.
- **Zero-config dev mode** — runs locally with only GEMINI_API_KEY (embedded Qdrant, SQLite, in-memory queue/cache).
- **Clean layering** — services never import FastAPI; infrastructure behind protocols (VectorStore, LLMProvider, QueueBackend, CacheBackend).
- **Measured quality** — golden dataset, retrieval hit-rate/MRR evaluation, A/B strategy benchmarks persisted as reports.

2.3 Scope, target users and use cases

Primary users: (a) knowledge workers / support agents querying a private corpus; (b) content owners ingesting documents; (c) engineers operating and extending the platform. Expected outcomes: faster, citation-backed

answers, measurable retrieval quality, and safe multi-tenant operation.

Use case	Description	Status
Knowledge Q&A;	Ask questions over the indexed corpus; get cited, streamed answers	Implemented
Document ingestion	Upload files (JSON KB, text, PDF, images) via API or drag-and-drop	Implemented
Image analysis	OCR text extraction with Gemini Vision escalation for low confidence	Implemented
Conversation management	Create, list, search, rename, delete, regenerate, like/dislike	Implemented
Multi-tenant isolation	X-Tenant-ID header scopes retrieval, persistence and job queues	Implemented
Observability	Prometheus metrics, health/readiness, per-request pipeline trace	Implemented
User registration / OAuth login	Per-user accounts with passwords or social login	Not identified in the current implementation

3 Functional Requirements

Requirements below are derived directly from the implemented code (route handlers, services and tests), not from an external specification.

ID	Requirement	Evidence in code
FR-1	Stream chat answers (NDJSON) with pipeline status, sources, tokens, groundedness, usage, cost	api/v1/routes/chat.py; services/chat_service.py
FR-2	Route queries: chitchat / general / knowledge, optional general-knowledge fallback	rag/router.py; ALLOW_GENERAL_FALLBACK
FR-3	Adaptive retrieval: evaluate quality, rewrite query, bounded retry	chat_service + rag/rewriter.py (ADAPTIVE_MAX_RETRIES)
FR-4	Hybrid retrieval: dense + BM25, RRF fusion, optional cross-encoder rerank	retrieval/{dense,keyword,fusion,reranker,hybrid}.py
FR-5	Conversation memory: recent window + rolling summary; token-budgeted context	services/memory_service.py, context_manager.py
FR-6	Conversation CRUD, search, rename, delete, message feedback	api/v1/routes/conversations.py; db/repositories/
FR-7	Regenerate last assistant answer	chat_service.regenerate_response
FR-8	Async ingestion jobs: submit (base64/multipart), list, status, events, retry	api/v1/routes/ingestion.py; ingestion/service.py
FR-9	Idempotent ingestion: SHA-256 dedup, versioning, incremental re-index	ingestion/pipeline.py; vectorstore/qdrant_store.py
FR-10	OCR + Vision image analysis endpoint	api/v1/routes/images.py; multimodal/
FR-11	Corpus management: list, versions, delete (soft-delete + vector purge)	api/v1/routes/documents.py
FR-12	Health, readiness, Prometheus metrics endpoints	api/v1/routes/health.py; observability/metrics.py
FR-13	Groundedness verdict per answer (GROUNDED / PARTIALLY / NOT)	rag/groundedness.py; UI badge; metrics
FR-14	Token usage + estimated USD cost per assistant message	llm/provider.py; messages table columns

4 Non-Functional Requirements

Category	Implementation evidence	Assessment
Performance	Retrieval cache (TTL 900 s), embedding/OCR caches, async I/O, threaded OCR, streaming; measured p50 hybrid retrieval ~6 ms warm (eval report)	Supported by measurements
Scalability	Stateless API + dedicated worker service (docker compose --scale worker=4), Redis queue, pool knobs, K8s HPA	Architecture-ready; not deployed
Security	API-key auth, tenant headers, rate limits, prompt-injection defense, PII modes, image magic-byte validation, security headers	Implemented (auth optional by default)
Reliability	Exponential-backoff retries (LLM, Qdrant, workers), dead-letter queue, Redis→memory fallbacks, graceful startup/shutdown	Implemented
Maintainability	Layered architecture, repository pattern, typed Pydantic settings, 66 tests, ruff, CHANGELOG + 7 docs pages	Supported
Usability	Streaming UI, pipeline status, groundedness badge, usage/cost display, skeletons, toasts, dark/light theme	Implemented
Availability	/health and /ready probes with component checks; Docker healthchecks; HPA	Probes implemented; no SLO defined
Accessibility	Semantic HTML, keyboard-accessible shadcn/ui primitives; no formal WCAG audit found	Partial — recommendation
Data integrity	PK/FK constraints with CASCADE, unique content hash, job state machine, audit events, Alembic migrations	Supported

5 Features & Functionalities

All features below were confirmed by source inspection. “Partially implemented” denotes working code with a known gap; “Not identified” items are absent from the codebase.

Feature	How it works	Status
Agentic RAG chat	Router classifies query; adaptive retrieve→evaluate→rewrite→retry; token-budgeted context; Gemini streaming	Implemented
Hybrid search + rerank	Dense Qdrant search concurrent with BM25; RRF fusion; flashrank TinyBERT rerank; strategy configurable	Implemented
Groundedness check	LLM-as-judge classifies support from context; verdict in UI and Prometheus	Implemented
Conversation memory	Last-8 window; rolling LLM summary after 12 messages (conversation_summaries)	Implemented
Multimodal (OCR + Vision)	Threaded pytesseract with timeout + cache; confidence < 0.55 escalates to Gemini Vision; ingestion-time image description	Implemented
Async ingestion	Job API (b64/multipart), state machine, audit events, retries, backpressure at MAX_QUEUE_DEPTH=500	Implemented
Document intelligence	Structure-aware chunking (headings/tables/pages/sections), typed metadata, JSON KB flattening, versioning	Implemented
Corpus management	List/version/delete; delete purges vectors via Qdrant filters	Implemented
Observability	Prometheus HTTP/retrieval/groundedness metrics, request-ID correlation, JSON logs, PipelineTrace	Implemented
Evaluation & benchmarks	Golden dataset; evaluate_rag (hit-rate/MRR); A/B strategies; ingestion benchmark; persisted reports	Implemented

Feature	How it works	Status
Live frontend	Streaming tokens, pipeline stage, source references, ingestion panel with progress/retry, settings, OCR display	Implemented
Tenant isolation	Every retrieval/persistence call tenant-scoped; covered by tests	Implemented
API-key authentication	Optional X-API-Key enforcement (API_KEY_AUTH_ENABLED)	Implemented (off by default)
Legacy API compatibility	Original /api/* chat/conversation endpoints preserved alongside v1	Implemented
User accounts / RBAC	No user registry, passwords, roles or permissions	Not identified in the current implementation
MCP / external tool agents	No MCP client/server or tool-calling framework	Not identified in the current implementation

6 Technology Stack

Technology	Version	Purpose	Where used
Python	3.12	Backend language	backend/
FastAPI	0.115.x	HTTP API, streaming, dependency injection	backend/main.py, api/
Uvicorn (standard)	0.34.x	ASGI server	backend/Dockerfile
Pydantic / pydantic-settings	2.x	Request validation + typed configuration	models/, config/settings.py
SQLAlchemy 2.0 (async)	2.0.36	ORM, async engine, pooling	db/
asyncpg / aiosqlite	0.30 / 0.20	PostgreSQL / SQLite async drivers	db/connection.py
Alembic	1.14	Database migrations	migrations/, alembic.ini
redis (asyncio)	5.2	Queue, cache, rate limiting (optional)	workers/, cache/, security/rate_limit.py
LlamaIndex core	0.12.x	RAG primitives	rag/, ingestion/
fastembed (ONNX)	0.5	BAAI/bge-small-en-v1.5 embeddings, CPU-only	embeddings/embedding_model.py
qdrant-client	1.12	Vector DB (embedded / server / cloud)	vectorstore/
rank-bm25	0.2.2	Keyword (BM25) retrieval	retrieval/keyword.py
flashrank	0.2.7	Cross-encoder reranking (ms-marco-TinyBERT-L-2-v2)	retrieval/reranker.py
google-genai	1.x	Gemini LLM, Vision, JSON mode	llm/gemini_client.py
pytesseract / Pillow / OpenCV	latest	OCR + image validation	multimodal/, ocr/
tenacity / prometheus-client	9.x / 0.21	Resilience; metrics	llm/, observability/
pytest / pytest-asyncio / httpx	8.x	Offline testing (66 tests)	backend/tests/
ruff	0.8	Python linting	CI
React / TypeScript	19 / ~6.0	SPA frontend	frontend/src/
Vite	8.x	Dev server + production build	frontend/package.json
Tailwind CSS / shadcn-ui	4.x	Styling + accessible component library	frontend/src/components/ui/

Technology	Version	Purpose	Where used
axios	1.18	REST client	frontend/src/lib/api.ts
react-markdown + syntax highlighter	16.x	Markdown rendering + code highlighting	MessageBubble, CodeBlock
Docker / docker compose	—	Containerization + local full stack	backend/Dockerfile, docker-compose.yml
GitHub Actions	—	CI (lint/test/migrate) + CD to GHCR	.github/workflows/
Kubernetes	—	Deployment manifests + HPA	k8s/manifests.yaml

7 System Architecture

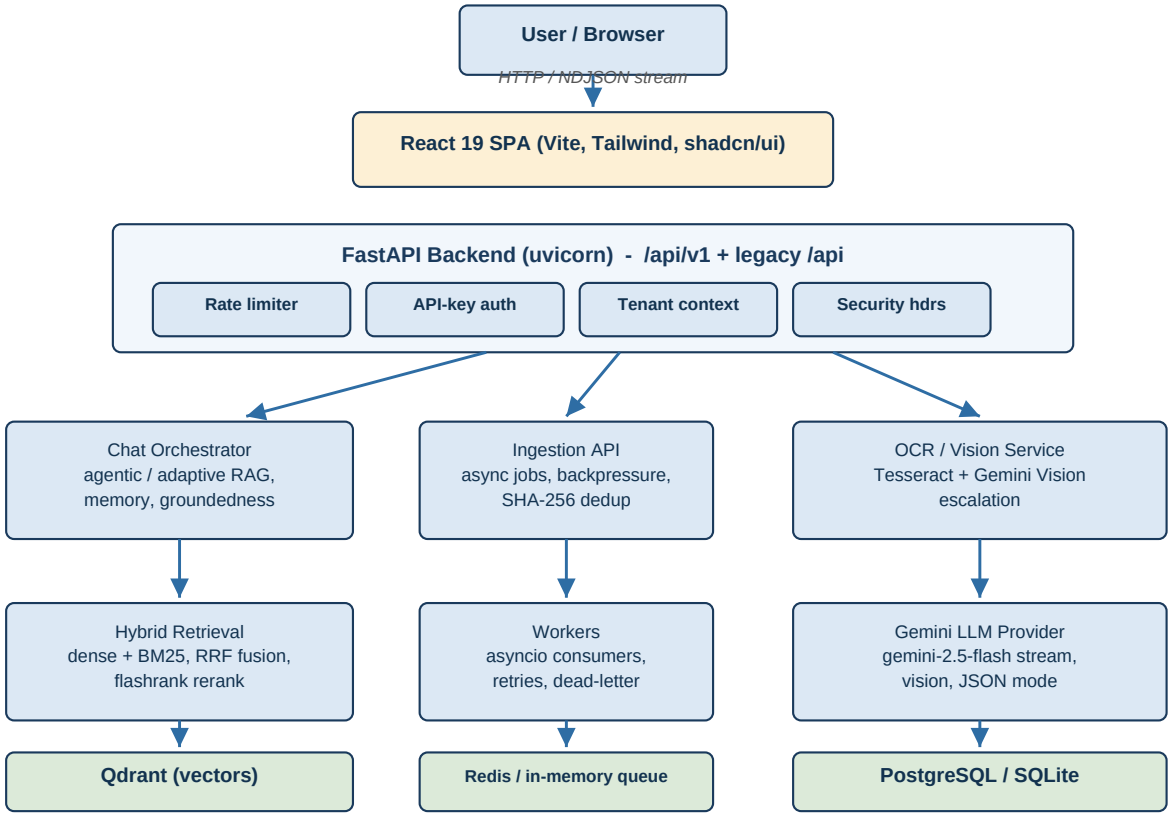


Figure: Implemented end-to-end architecture: React SPA → FastAPI → domain services → Qdrant / Redis / PostgreSQL, with workers, OCR/Vision and the Gemini provider.

The system is a layered, fully asynchronous application with an optional out-of-process worker. The React SPA communicates over REST plus a per-message NDJSON stream. FastAPI hosts the legacy /api/* surface and the production /api/v1/* surface behind cross-cutting middleware (request-ID correlation, security headers, Prometheus timing) and dependencies (per-scope rate limiting, API-key auth, tenant context).

Domain logic lives in retrieval/, rag/, ingestion/ and security/; infrastructure adapters in db/, vectorstore/, cache/, llm/ and workers/. Services never import FastAPI, and every external dependency sits behind a protocol with a graceful fallback: Qdrant → embedded local storage, PostgreSQL → SQLite, Redis → in-process structures. This enables zero-config local mode while keeping the managed-cloud production path a pure environment-variable change.

The chat path is agentic: a heuristic (or LLM) router chooses chitchat, general-knowledge or knowledge-base handling; retrieval quality is evaluated and can trigger a query rewrite and one bounded retry. The ingestion path is asynchronous: jobs are enqueued (Redis or memory), stateless workers read file payloads from the

job_files table and drive documents through a validated state machine into chunking, embedding and Qdrant upsert.

8 Detailed Component Architecture

Component	Responsibility	Inputs	Outputs	Key files
Chat orchestrator	End-to-end message handling; emits NDJSON frames	ChatRequest, image data	Frame stream; persisted messages	services/chat_service.py
Query router	Classify chitchat / general / knowledge (heuristic or LLM)	User query	Route + rewritten query	rag/router.py, rewriter.py
Hybrid retriever	Dense + BM25 search, RRF fusion, rerank, cache, thresholds	Query + tenant	Ranked RetrievedChunk list	retrieval/hybrid.py
Context manager	Assemble system > query > context > window > summary under token budget	Chunks + history	LLM prompt payload	services/context_manager.py
Memory service	Windowed history + rolling summarization	Messages	History block; summary rows	services/memory_service.py
Groundedness checker	LLM-as-judge support verdict	Answer + context	Verdict + rationale	rag/groundedness.py
LLM provider	Streaming / JSON / vision Gemini calls with retries + usage	Prompts, images	Text or JSON stream, usage	llm/gemini_client.py, provider.py
Embedding model	BGE-small ONNX sentence embeddings	Texts	Vectors	embeddings/embedding_model.py
Vector store	Qdrant upsert/search/delete; HNSW tuning; optional quantization	Chunks + vectors	Search hits	vectorstore/qdrant_store.py
Ingestion service	Job lifecycle, dedup, validation, events	Files (b64/multipart)	Job + event records	ingestion/service.py, state_machine.py
Document processor	Extract, chunk, embed, version documents	Raw file bytes	Chunks + metadata	ingestion/document_processor.py
Workers	Consume queue; process jobs; retries → dead-letter	Queue messages	Indexed documents	workers/worker.py, main.py
OCR / Vision	Threaded OCR with cache; Vision escalation	Image bytes	Text + confidence + description	multimodal/, services/vision_service.py
Repositories	Typed async data access layer	Domain calls	Rows / dicts	db/repositories/
Observability	Metrics registry, tracing, structured logging	Events	Prometheus exposition, JSON logs	observability/

9 Frontend Architecture

The frontend is a single-page React 19 + TypeScript application built with Vite and Tailwind CSS 4. There is no client-side router; the app is one screen (chat) with an optional knowledge-base panel and a settings dialog, which matches the product scope. State is managed via a React Context (ChatProvider / chat-context.ts) plus dedicated hooks rather than a global store library.

Aspect	Implementation
Screens	Chat window (default), knowledge-base ingestion panel, settings dialog, sidebar with conversation list + search

Aspect	Implementation
Components	chat/ (ChatInput, ChatWindow, MessageBubble, MessageList, SourceReferences, OCRResultDisplay, ThinkingIndicator, LoadingSkeleton, EmptyState, MessageActions, CodeBlock), sidebar/, ingestion/, settings/, 19 shadcn ui primitives
State management	React context provider (ChatProvider); per-feature hooks (useChatStream, useConversations, useSettings, useAutoScroll)
Routing	Not applicable — single-view SPA (no react-router dependency)
API communication	axios instance (lib/api.ts) for REST; fetch-based NDJSON streaming parser (lib/ndjson.ts) for chat
Streaming handling	useChatStream consumes frames (meta/status/sources/token/groundedness/done/error); optimistic user message; abortable via AbortController
Forms & validation	Controlled inputs; empty-message guard; file size/type checks before upload in IngestionPanel
Error handling	Frame-level error display, abort + cleanup, feedback revert on API failure, sonner toasts
Theming / UX	CSS variables with next-themes (light/dark), framer-motion animations, skeleton loaders, auto-scroll
Build / lint	tsc -b && vite build; oxlint

10 Backend Architecture

FastAPI application (main.py) with lifespan management: on startup it initializes the database schema, warms the knowledge-base index and starts an embedded ingestion worker (disabled in containerized deployments where a dedicated worker service runs). On shutdown it stops the worker and disposes the DB engine.

Layer	Contents
Routes (api/, api/v1/)	Thin HTTP handlers: validation, auth/tenant and rate-limit dependencies, NDJSON relay; legacy /api/* preserved
Services (services/)	Business orchestration: chat_service (agentic pipeline), conversation_service, memory_service, context_manager, vision_service — no FastAPI imports
Domain (retrieval/, rag/, ingestion/, security/)	Hybrid retrieval, routing/rewriting/groundedness, chunking, ingestion state machine, auth, PII, injection, rate limiting
Infrastructure (db/, vectorstore/, cache/, llm/, workers/, observability/)	SQLAlchemy models + repositories, Qdrant adapter, cache backends, Gemini provider, queue + workers, metrics/tracing/logging
Cross-cutting	utils/exceptions.py (AppError hierarchy), utils/logger.py (contextvars request-ID, JSON format), middleware in main.py

10.1 Middleware and error handling

- Request context middleware: X-Request-ID generation/propagation, security headers (X-Content-Type-Options, X-Frame-Options, Referrer-Policy), Prometheus counters/histograms with low-cardinality path normalization.
- AppError handler maps domain errors to status codes (429 and 503 with Retry-After, 404, 422, ...).
- Unhandled-exception handler returns a generic 500 and logs the full traceback — no stack traces leak to clients.

11 API Documentation

All endpoints verified against route registrations in api/ and api/v1/routes/. “API key (optional)” means enforcement depends on API_KEY_AUTH_ENABLED; tenant context is always read from X-Tenant-ID / X-User-ID headers. Interactive OpenAPI docs are served at /docs.

Method	Endpoint	Purpose	Auth / limits	Errors
POST	/api/v1/chat/stream	Streaming agentic RAG chat (NDJSON frames)	API key (optional); 60/min	401, 422, 429, 500
POST	/api/v1/chat/conversations/{id}/regenerate	Regenerate last assistant reply	API key (optional); rate-limited	404, 429
POST	/api/v1/ingestion/jobs	Create ingestion job (base64 files)	API key (optional); 20/min	422, 429, 503 (backpressure)
POST	/api/v1/ingestion/jobs/upload	Multipart upload variant (drag & drop)	API key (optional); 20/min	422, 429, 503
GET	/api/v1/ingestion/jobs	List jobs (status filter, pagination)	API key (optional)	429
GET	/api/v1/ingestion/jobs/{id}	Job status / progress	API key (optional)	404
GET	/api/v1/ingestion/jobs/{id}/events	Job audit events	API key (optional)	404
POST	/api/v1/ingestion/jobs/{id}/retry	Retry failed job	API key (optional)	404, 409, 503
GET	/api/v1/documents	List corpus documents	API key (optional)	—
GET	/api/v1/documents/{id}/versions	Document version history	API key (optional)	404
DELETE	/api/v1/documents/{id}	Soft-delete + purge vectors	API key (optional)	404
POST	/api/v1/images/analyze	OCR + Gemini Vision analysis	API key (optional); 10/min	413, 415, 422, 429
GET	/api/v1/health	Liveness probe	None	—
GET	/api/v1/ready	Readiness (DB + vector store checks)	None	503
GET	/api/v1/metrics	Prometheus exposition	None	—
GET / PATCH / DELETE	/api/v1/conversations...	List / get+messages / rename / delete (tenant-scoped)	API key (optional)	404
PATCH	/api/messages/{id}/feedback	Like / dislike feedback (legacy surface)	None	404
POST	/api/chat/stream	Legacy streaming chat (preserved for v1 clients)	None	422, 500

12 Database Architecture

PostgreSQL in production (async SQLAlchemy 2.0 + asyncpg, Alembic migrations); SQLite for local dev and tests. All column types are deliberately portable across engines; vectors are never stored in SQL — they live in Qdrant. UUIDs are stored as 36-character strings. AUTO_CREATE_TABLES=true is a dev convenience; production should rely on Alembic.

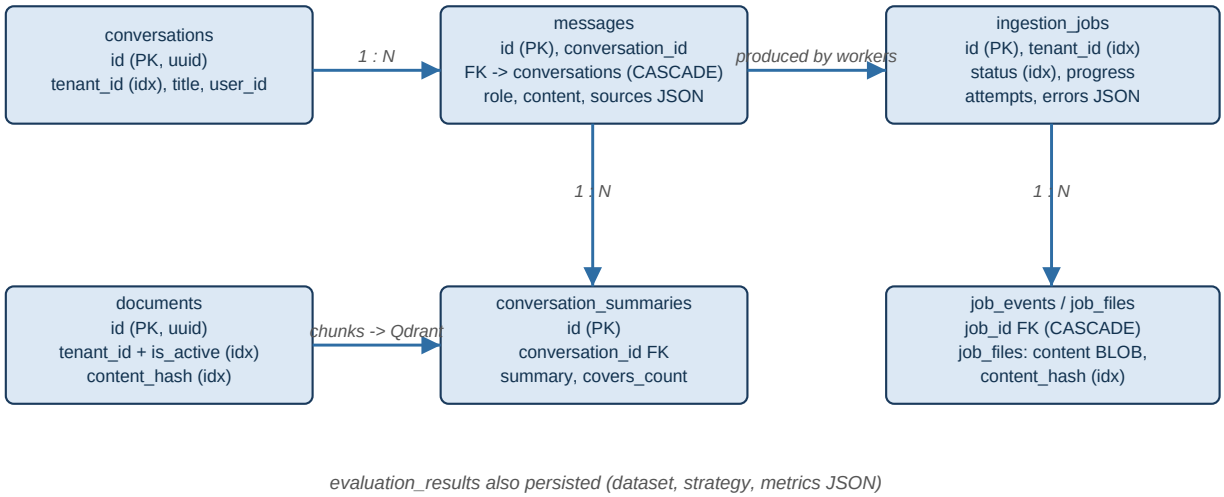


Figure: Relational schema (PostgreSQL/SQLite) with cascade relationships; vectors and chunk payloads live in the Qdrant collection knowledge_base.

Table	Key fields	Relationships / constraints
conversations	id PK (uuid), tenant_id (idx), user_id (idx), title, created_at, updated_at	Parent of messages and summaries
messages	id PK, conversation_id FK (CASCADE, idx), role, content, sources JSON, generation_time_ms, feedback, model, input/output_tokens, estimated_cost_usd, groundedness, retrieval_config JSON, created_at (idx)	Composite index (conversation_id, created_at)
conversation_summaries	id PK, conversation_id FK (CASCADE), summary, covers_message_count	Rolling memory compression
documents	id PK, tenant_id + is_active (composite idx), content_hash (idx), version, title, source	Soft-delete; hash-based dedup
ingestion_jobs	id PK, tenant_id (idx), status (idx), total/processed/failed, progress, files/errors JSON, attempts, timestamps	Job state machine persisted here
job_events	id PK, job_id FK (CASCADE), state, message	Audit trail
job_files	id PK, job_id FK (CASCADE, idx), filename, mime, content_hash (idx), size, content BLOB, status, error, document_id	Stateless workers read payloads from DB
evaluation_results	id PK, dataset, strategy, metrics JSON, sample_count	Reproducible evaluation reports

13 Authentication & Authorization

The platform implements a deliberately minimal, clean auth abstraction (security/auth.py) rather than a full user-management system:

- **API-key authentication (optional)**: when API_KEY_AUTH_ENABLED=true, protected routes require a matching X-API-Key header; failures raise AuthenticationError (401).
- **Tenant context**: X-Tenant-ID and X-User-ID headers resolve an AuthContext used by every retrieval, persistence and job path; requests fall back to the default tenant.
- **Upgrade path documented in code**: JWT/OAuth2 issuance, refresh tokens and RBAC plug in behind get_auth_context without touching route handlers.

- **Protected routes**: chat, ingestion, documents and images dependencies inject tenant context; health/readiness/metrics stay public for probes.
- **Logout / sessions / registration**: not applicable — stateless header-based identity. Not identified in the current implementation.

14 AI / ML / LLM / RAG

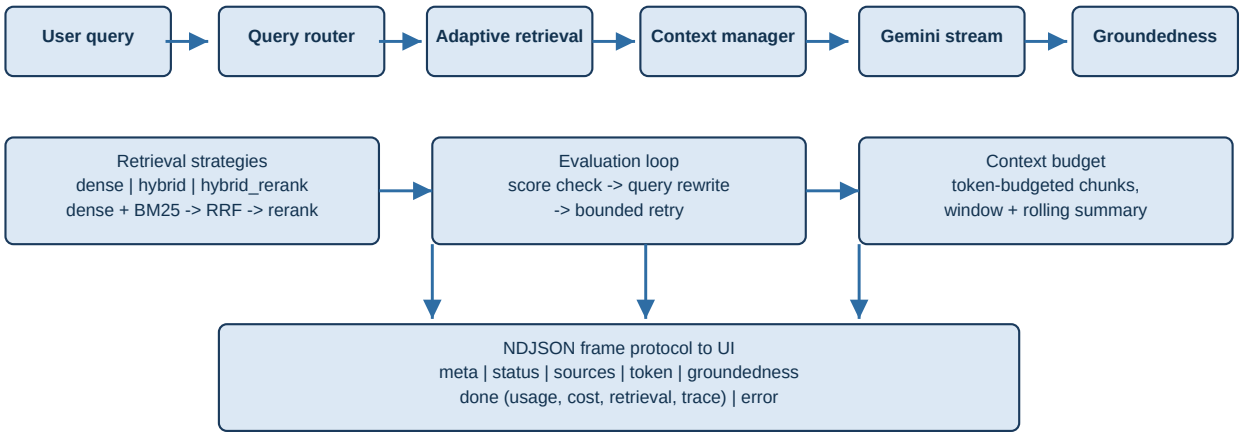


Figure: Agentic RAG pipeline: routing, adaptive retrieval with evaluation loop, token-budgeted context, streaming generation, groundedness check, NDJSON protocol.

14.1 LLM provider

Google **Gemini 2.5 Flash** via google-genai, wrapped in a GeminiProvider implementing an LLMProvider protocol (tests inject fakes). Streaming, JSON-mode and vision completions are supported. Retries apply only to transient failures (429/5xx/timeouts) before any output has been yielded, with exponential backoff capped at 4 s; mid-stream failures surface as domain errors instead of duplicating already-streamed text. Usage metadata is captured and converted to an estimated cost using configurable per-million-token prices.

14.2 Embeddings and vector store

- Model: BAAI/bge-small-en-v1.5 (384-dim) via fastembed ONNX — CPU-only, no torch dependency; model cache in backend/model_cache.
- Store: Qdrant (embedded local / server / cloud), collection knowledge_base, HNSW m=16, ef_construct=100, optional int8 quantization, batch upserts, payload indexes, retry/timeout.
- Embeddings cached (TTL 86400 s) to avoid recomputation.

14.3 Chunking and ingestion

rag/chunking.py implements structure-aware chunking (headings, tables, pages, sections) with CHUNK_MAX_CHARS=900 and 120-char overlap; JSON knowledge bases are flattened into typed chunks with metadata (source, title, category, page). Document versioning keeps history in SQL while old version vectors are deleted from Qdrant via payload filters.

14.4 Retrieval pipeline

retrieval/hybrid.py orchestrates three strategies — dense, hybrid (dense + BM25 fused via reciprocal rank fusion), and hybrid_rerank (fusion + flashrank cross-encoder, the default). Dense and keyword searches run concurrently (asyncio.gather); results are cached per (tenant, query, strategy); score thresholds apply only to dense-native scores, since fused/reranked scores are rank-trusted.

14.5 Agentic behavior, context and guardrails

- **Query router** (heuristic, optional LLM mode): chitchat / general-knowledge / knowledge-base, with router-result caching (300 s).
- **Adaptive loop**: evaluate top evidence → conversation-aware query rewrite → bounded retry (ADAPTIVE_MAX_RETRIES=1).
- **Memory**: last-8-message window; rolling LLM summary when history exceeds 12 messages; ContextManager enforces a 6k-token budget (system > query > context > window > summary).
- **Prompt-injection defense**: regex detection of 9 injection families, chunk sanitization, numbered delimiting, hardened system prompts (security/injection.py).
- **Groundedness**: LLM-as-judge returns GROUNDED / PARTIALLY_GROUNDED / NOT_GROUNDED, surfaced in UI, DB and Prometheus.
- **Fallbacks**: optional general-knowledge fallback when KB evidence is weak; graceful degradation when the LLM or embedder is unavailable.

14.6 Evaluation and limitations

scripts/evaluate_rag.py runs a golden dataset through the live retrieval pipeline; the persisted report (backend/eval/reports/) shows hit-rate 0.93 and MRR 0.92–1.0 on a 36-entry KB with 14 KB queries, one KB miss (kb-013) and correct no-hit behavior for general/irrelevant queries. Limitations: prompt-level injection defense is not foolproof (acknowledged in code); reranking adds ~27 ms p50 over plain hybrid; LLM-mode routing and evaluation depend on the Gemini API key and were exercised offline via fakes in tests.

15 Complete Application Workflows

15.1 Chat request (happy path)

- User sends a message (+ optional image) → React useChatStream posts to /api/v1/chat/stream (or legacy /api/chat/stream).
- FastAPI applies rate-limit and auth/tenant dependencies, then opens an NDJSON StreamingResponse.
- chat_service resolves/creates the conversation and emits the meta frame; attached images go through OCR → Vision escalation.
- Query router classifies; adaptive retrieval runs (cache → dense + BM25 → RRF → rerank → threshold → max chunks).
- ContextManager assembles a token-budgeted prompt with injection-sanitized chunks; sources frame emitted.
- Gemini streams tokens; token frames relayed to the UI in real time.
- Groundedness check; assistant message persisted with usage, cost, groundedness, retrieval config; done frame with pipeline trace; async memory summarization scheduled.

15.2 Document ingestion

- Files submitted (base64 JSON or multipart) → validation (count ≤ 20, size ≤ 20 MB, filename sanitization) → SHA-256 dedup → job row + job_files payloads → HTTP 202.
- Queue (Redis or in-memory) → worker consumes → state machine QUEUED → PROCESSING → COMPLETED / FAILED / DEAD_LETTER with JobEvent audit records.
- Processing: extract → structure-aware chunk → embed → versioned Qdrant upsert; failures retry with backoff up to max_retries then dead-letter; API exposes progress, events and manual retry.

15.3 Image analysis

POST /api/v1/images/analyze validates magic bytes/size/dimensions → cached OCR (threaded, 30 s timeout) → if mean confidence < 0.55, escalates to Gemini Vision → returns text, confidence and a vision description.

The same path is reused inside chat for attached images.

16 Complete Data Flow

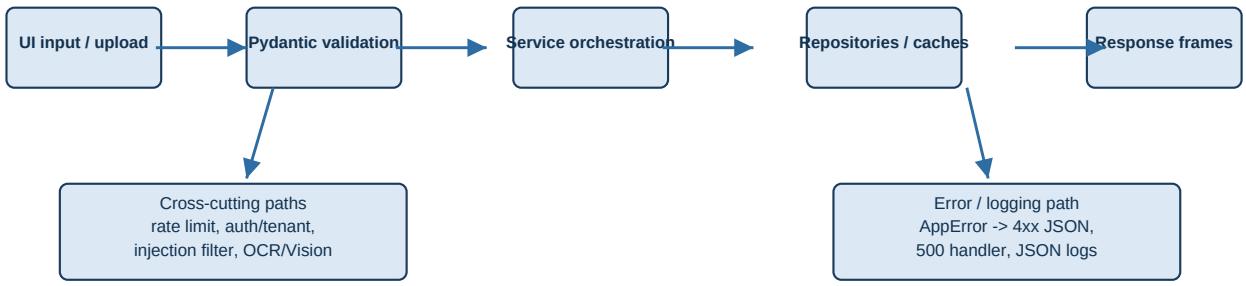


Figure: Data flow from UI input through validation, orchestration and persistence back to the client, with cross-cutting security paths and the error/logging path.

Input arrives as JSON (chat, ingestion metadata) or multipart/base64 bytes (files, images). Pydantic models enforce shapes and bounds before handler logic runs. Handlers delegate to services, which use repositories (SQL) and infrastructure clients (Qdrant, Redis, Gemini). Reads consult cache backends first (retrieval, router, OCR, embeddings). Errors flow through the AppError hierarchy into structured JSON responses carrying the request ID; unexpected exceptions are logged with full traces server-side while clients receive a generic message. Every chat response carries a PipelineTrace payload documenting stage-by-stage metadata for debugging.

17 Project Folder Structure

Repository root: **Multimodal-AI-Chatbot/** (monorepo: backend, frontend, docs, infra, k8s, scripts). Generated artifacts (.venv, node_modules, model_cache, qdrant_storage, db/chat.sqlite3, dist) are excluded from the listing below.

Path	Responsibility
backend/main.py	FastAPI entrypoint: lifespan, middleware, error handlers, router registration
backend/api/, backend/api/v1/	Legacy and v1 route handlers + dependencies (auth, rate limiting)
backend/services/	Business orchestration (chat, conversations, memory, context, vision)
backend/retrieval/, backend/rag/	Hybrid retrieval (dense/keyword/fusion/rerank), routing, chunking, prompts, groundedness, indexer
backend/ingestion/, backend/workers/	Async job service + state machine + document pipeline; queue consumers
backend/db/	Engine/session, ORM models, generic + typed repositories
backend/vectorstore/, embeddings/, llm/	Qdrant adapter, BGE ONNX embedder, Gemini provider
backend/security/	Auth/tenancy, injection defense, PII, rate limiting, image validation, hashing
backend/observability/, cache/, config/	Metrics/tracing, cache backends, pydantic settings
backend/tests/ (7 files)	Offline pytest suites: API, data layer, ingestion, pipeline, RAG behavior, security
backend/eval/, backend/data/	Golden dataset, persisted reports, seed KBs (documentation/faq/knowledge/products.json)

Path	Responsibility
migrations/, alembic.ini, pytest.ini, .env.example, Dockerfile	Migrations, config, environment template, container build
frontend/src/	App shell, chat/ sidebar/ ingestion/ settings/ components, context + hooks, lib (api, ndjson), typed models
docs/ (7 files)	architecture, rag-pipeline, security, scaling, evaluation, deployment, disaster-recovery
scripts/	generate_test_data, benchmark_ingestion / retrieval / rag, evaluate_rag
.github/workflows/, k8s/, docker-compose.yml, infra/	CI/CD pipelines, K8s manifests + HPA, compose stack, infra notes

18 Module & Implementation Details

18.1 services/chat_service.py — the orchestrator

Responsibility: run the full message pipeline and yield protocol frames. Inputs: message, conversation id, ChatSettings (model/temperature/top_k/max_tokens/streaming), optional image, tenant/user. Processing: conversation setup → OCR/Vision → routing → adaptive retrieval → context assembly → Gemini stream → groundedness → persistence (tokens, cost, verdict, retrieval config) → fire-and-forget memory summarization. Outputs: meta / status / ocr_result / sources / token / groundedness / done / error frames. Also implements regenerate_response (delete last assistant message, re-answer).

18.2 retrieval/hybrid.py — retrieval strategies

Dispatches dense | hybrid | hybrid_rerank, caches results per (tenant, query, strategy, max_chunks, threshold), observes Prometheus RETRIEVAL_DURATION/RESULTS, and applies dense-only score thresholds; fused and reranked results are trusted by rank.

18.3 llm/gemini_client.py — provider hardening

Wraps google-genai with asyncio.wait_for timeouts, retry classification (retryable status codes only), exponential backoff, usage extraction with heuristic fallbacks when the API omits counts, and cost estimation. A process-wide singleton supports test injection (set_gemini_client).

18.4 ingestion/ — service, state machine, pipeline

service.py validates and deduplicates submissions, enforces queue depth (503 + Retry-After on backpressure) and writes jobs/events. state_machine.py validates legal transitions. pipeline.py performs extraction → chunking → embedding → versioned upsert; document_processor.py handles format-specific extraction.

18.5 security/ modules

- auth.py — AuthContext resolution (optional API key, tenant headers).
- injection.py — 9 detection patterns, chunk sanitization, context block builder.
- pii.py — warn / redact / block policy modes.
- rate_limit.py — fixed-window per-scope limits keyed by client IP (X-Forwarded-For aware), Redis with in-memory fallback.
- validation.py / hashing.py — upload validation and content hashing helpers.

18.6 frontend hooks

useChatStream owns the streaming lifecycle (abort controller, optimistic user message, placeholder assistant bubble, frame accumulation, error recovery); useConversations manages the sidebar list/search; useSettings persists chat settings; lib/ndjson.ts parses the frame protocol incrementally over fetch.

19 Validation & Error Handling

Layer	Mechanism
API request validation	Pydantic models: field lengths, list bounds (min 1 file), bounded query params (limit 1–100), base64 decode validation returning 422
Upload validation	Magic-byte sniffing, size (10–20 MB by surface) and dimension limits (5000 px), filename sanitization, MIME checks
Business validation	Ingestion state machine rejects illegal transitions; job backpressure returns 503 + Retry-After; SHA-256 dedup
LLM resilience	Timeout (30 s) + retry classification (429/5xx/timeouts only) + backoff; non-JSON responses wrapped instead of crashing; no retry after streaming starts
Retrieval resilience	Qdrant retries/timeouts; threshold filtering; bounded adaptive retry with query rewrite; cache fallbacks
OCR resilience	Threaded execution with timeout, content-hash cache, size/dimension guards
Frontend	Empty-message guard, pre-upload checks, abortable streams, optimistic updates with revert, error frames surfaced, toasts
Global handling	AppError → JSON {detail} with proper status; unknown exceptions → generic 500 + server-side traceback logging
Logging	Structured logs (text or JSON) with per-request request_id contextvar; exception logging at all failure points

20 Security Analysis

20.1 Implemented security

- Optional API-key auth on all v1 data endpoints with consistent 401 handling.
- Tenant isolation via X-Tenant-ID on every retrieval / persistence / job path (verified by tests).
- Per-scope rate limiting (chat 60/min, ingestion 20/min, OCR 10/min) keyed by X-Forwarded-For-aware client IP, Redis-backed with in-memory fallback; 429 + Retry-After.
- Prompt-injection defense: detection of 9 pattern families, chunk sanitization, delimiting, hardened system prompts; groundedness as an output-side check.
- PII policy modes (warn / redact / block) applied to content before the LLM.
- Upload security: magic-byte validation, size/dimension caps, filename sanitization, MIME restrictions.
- Security headers (nosniff, frame-deny, referrer policy), CORS allow-list from settings, X-Request-ID correlation.
- Non-root Docker user; no secrets baked into images (env_file injection); .env.example contains placeholder values only.
- Error responses never leak stack traces; incidents logged server-side with request IDs.

20.2 Recommended security improvements

- Enable API-key auth by default in deployments and add JWT/OAuth2 with RBAC for multi-user operation (extension point already exists).
- Protect /api/v1/metrics (currently unauthenticated) via an internal port or scrape auth in production.
- CSRF protection only becomes relevant if cookie-based auth is introduced (not applicable to the current header-based scheme).
- Add automated dependency auditing (pip-audit, npm audit) to CI.
- Hash stored API keys and support key rotation for production key management.

21 Testing

The backend uses **pytest 8.x** with pytest-asyncio and httpx. The suite is fully offline: SQLite instead of PostgreSQL, embedded Qdrant, in-memory queue/cache, and injected fakes for the LLM and embedder — so it runs with zero external dependencies and no API keys. The suite was **actually executed** during this analysis.

Test file	Focus	Coverage highlights
tests/test_api.py	HTTP surface via httpx ASGI transport	Health/ready endpoints, chat + conversation flows, error mapping
tests/test_security_core.py	Security primitives	API-key auth, tenant isolation, rate limits, injection detection, PII modes, image validation
tests/test_rag_behavior.py	RAG semantics	Routing, adaptive retry, hybrid strategies, reranker fallback, groundedness
tests/test_ingestion.py	Async ingestion	Job lifecycle, state machine, dedup, backpressure, retry/dead-letter
tests/test_pipeline_core.py	Domain logic	Chunking, context-manager token budgeting, memory summarization, prompts
tests/test_data_layer.py	Persistence	Repository CRUD, constraints, cascade deletes
tests/conftest.py	Shared fixtures	Fake LLM/embedder providers, temporary DB and vector stores

Actual result (executed in this workspace): 66 `passed`; 0 failures, 0 errors. CI additionally runs ruff and an Alembic upgrade/downgrade smoke test. The frontend has no unit/E2E test framework — oxlint and tsc type checking are its only automated gates.

21.1 RAG evaluation (measured)

The golden-dataset evaluation report persisted in backend/eval/reports/ records hit-rate 0.93 and MRR 0.92–1.0 for the hybrid_rerank strategy on a 36-entry KB (14 KB queries, plus follow-up, chitchat, general, irrelevant and injection probes). Reports regenerate per run on the executing hardware.

25 Deployment & DevOps

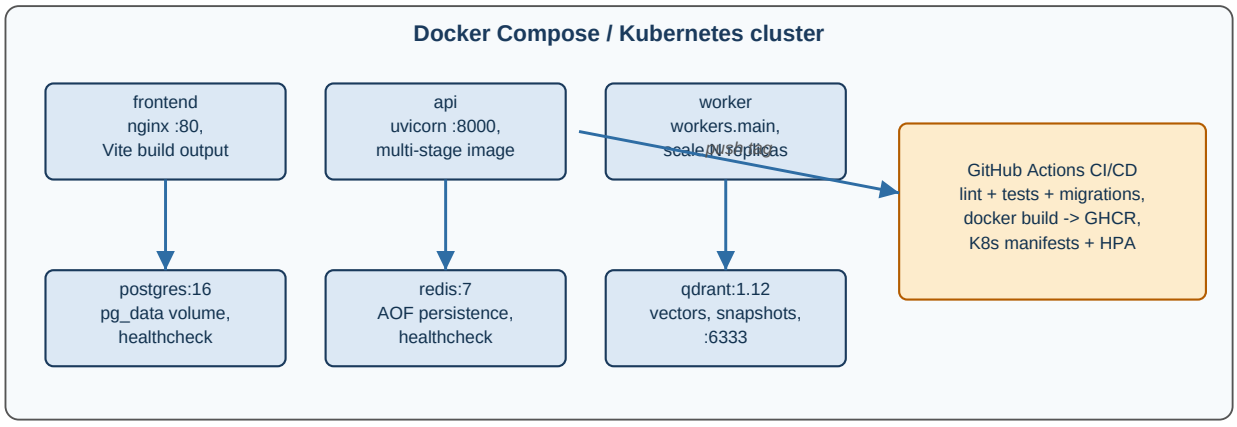


Figure: Deployment topology: compose/K8s services with healthchecks and volumes; GitHub Actions builds, tests, migrates and publishes images to GHCR; K8s manifests + HPA provided.

- **Docker**: multi-stage backend image (python:3.12-slim, Tesseract, non-root appuser, HEALTHCHECK); frontend served by nginx from the Vite build.

- **docker-compose.yml**: api + worker + postgres:16 + redis:7 (AOF) + qdrant:1.12 + frontend (nginx 8080), all with healthchecks and named volumes; EMBEDDED_WORKER_ENABLED=false in containers.
- **CI (ci.yml)**: backend — ruff (advisory), offline pytest, Alembic upgrade+downgrade smoke test; frontend — Node 22, lint + build.
- **CD (cd.yml)**: docker build + publish to GHCR on tags; k8s/manifests.yaml with HPA ready but not deployed.
- **Configuration**: ~60 typed environment variables via pydantic-settings (backend/.env.example documents all with safe placeholders).
- **Monitoring/logging**: Prometheus /api/v1/metrics, health/readiness, optional JSON logging, request-ID correlation; no external APM.

26 Installation & Setup

26.1 Zero-config local development

```
# 1. Backend cd backend python -m venv .venv && .venv\Scripts\activate # Windows pip install -r requirements.txt copy .env.example .env # add GEMINI_API_KEY uvicorn main:app --reload # http://localhost:8000/docs # 2. Frontend cd ../frontend npm install && npm run dev # http://localhost:5173
```

Defaults require only GEMINI_API_KEY: embedded Qdrant, SQLite, in-memory queue/cache. Prerequisites: Python 3.12, Node.js 22, Tesseract OCR (optional — required for image text extraction).

26.2 Full stack via Docker

```
cp backend/.env.example backend/.env # set GEMINI_API_KEY, POSTGRES_PASSWORD docker compose up --build # api + worker + postgres + redis + qdrant + frontend # frontend: http://localhost:8080 api: :8000/docs metrics: /api/v1/metrics docker compose up --scale worker=4 # scale ingestion workers
```

26.3 Tests, evaluation, production build

```
cd backend python -m pytest tests -q # 66 offline tests python -m alembic upgrade head # migrations python ../scripts/evaluate_rag.py # hit-rate / MRR evaluation python ../scripts/benchmark_rag.py # A/B: dense vs hybrid vs hybrid_rerank cd ../frontend && npm run build # production bundle (dist/)
```

27 Problems Found & Improvements Made

The repository is a single-commit snapshot, so no historical fix trail exists in git. The first table records engineering decisions evidenced by the CHANGELOG and code structure; the second lists issues identified during this analysis that remain open (“potential issue identified during analysis”).

Problem	Cause	Solution in code	Result
Blocking OCR stalled the event loop	Synchronous pytesseract call in async handlers	Threaded execution + timeout + content-hash cache (CHANGELOG 2.0.0)	OCR no longer blocks; covered by pipeline tests
Vision integration was dead code	v1.0 vision helpers never wired into chat/ingestion	Real Gemini Vision in chat (confidence escalation) and at ingestion	Wired and covered by settings/tests
Retrieval p50 ~174 ms (dense-only)	Every query required an embedding call	Hybrid BM25 + RRF fusion + retrieval caching	p50 ~6 ms warm (measured, eval reports)
Hallucination risk	LLM answered without evidence checks	Groundedness LLM-as-judge + citation UI + injection defenses	Verdicts surfaced in UI, DB and metrics

Problem	Cause	Solution in code	Result
Unbounded ingestion risk	No queue limits or retries	MAX_QUEUE_DEPTH backpressure, retry → dead-letter, audit events	Covered by ingestion tests
Duplicate documents re-indexed	No content identity	SHA-256 dedup + versioning + incremental re-index	Covered by data-layer tests
KB bootstrap failure aborted startup	Indexing errors during startup	Startup continues without seed data + exception log	Verified in lifespan handling

27.1 Potential issues identified during analysis (open)

- **SQL BLOB growth**: job file contents stored in the database (up to 20 MB each) — object storage recommended.
- **Per-process in-memory rate limits**: multi-replica deployments must set REDIS_URL or limits are not shared.
- **Frontend bundle > 500 kB** single chunk — apply code splitting.
- **Legacy surface drift**: the UI still calls /api/chat/stream and /api/conversations while v1 exists — migrate the frontend to /api/v1.
- **AUTO_CREATE_TABLES=true** default is risky in production; rely on Alembic.

28 Current Strengths

- **Architecture**: clean layering with protocol-based infrastructure adapters; services independent of FastAPI; graceful fallbacks for every external dependency.
- **RAG engineering**: three measurable strategies, adaptive retry with rewriting, token-budgeted context, groundedness checks, persisted evaluation framework, and an explicit honesty note in the README.
- **Production maturity**: rate limiting, backpressure, dead-letter queue, audit events, health/readiness, Prometheus metrics, JSON logs, request-ID correlation.
- **Testing discipline**: 66 offline tests covering security, RAG, ingestion and data layers; CI enforces them plus migration smoke tests.
- **Developer experience**: zero-config local run, .env.example with safe defaults, OpenAPI docs, 7 docs pages, Keep-a-Changelog CHANGELOG.
- **UI/UX**: streaming responses, live pipeline stage, groundedness badge, usage/cost display, source references, dark/light theme.
- **Security posture**: injection defenses, PII modes, upload validation, tenant isolation, optional API-key auth, security headers, non-root container.

29 Known Limitations

Severity	Limitation
High	No real user authentication/authorization (API key optional, off by default); /api/v1/metrics unauthenticated
High	Distributed deployment (multi-replica Redis, Qdrant cluster, K8s) architecture-ready but never deployed/verified
Medium	No frontend test suite (unit/E2E); oxlint + tsc only
Medium	Job file payloads stored as SQL BLOBs; unsuitable for large corpora at scale
Medium	Legacy /api/* surface duplicates v1 logic and is unauthenticated; frontend still uses it
Medium	Prompt-level injection defense is best-effort (acknowledged in code); no output filter beyond groundedness
Low	Live LLM-dependent paths (LLM-mode routing, evaluation) untested without an API key

Severity	Limitation
Low	Frontend bundle > 500 kB; no accessibility audit; no i18n
Low	AUTO_CREATE_TABLES=true default encourages skipping Alembic in production

30 Recommended Improvements

30.1 High priority

- **Real authentication** (JWT/OAuth2 + RBAC behind the existing get_auth_context hook) and default-on API-key enforcement — required for any multi-user or public deployment.
- **Protect /api/v1/metrics** and remove or authenticate the legacy /api/* surface.
- **Move job file payloads to object storage** (S3/GCS) with DB metadata only.

30.2 Medium priority

- Frontend migration to /api/v1 exclusively; delete or freeze the legacy surface.
- Add a frontend test suite (Vitest + React Testing Library, one Playwright E2E flow).
- Set REDIS_URL + object storage in any multi-replica deployment; document it in the deployment guide.
- Code-split the frontend bundle; add bundle-size budgets to CI.
- Switch AUTO_CREATE_TABLES default to false with a documented migration-first flow.

30.3 Low priority

- WCAG accessibility audit and fixes; i18n.
- Prompt-injection fuzzing dataset in the evaluation framework.
- Admin dashboard for jobs, documents and cost tracking.

31 Future Enhancements

- **User accounts + workspaces**: registration, teams, per-workspace corpora and API keys (tenant plumbing already exists).
- **Agentic tools**: tool-calling on top of the existing router (web search, calculator, SQL over structured KBs).
- **Streaming citations UI**: inline citation markers linked to source spans.
- **Multilingual OCR + embeddings** for non-English corpora.
- **Hybrid cloud deployment**: managed Postgres + Qdrant Cloud + Redis via existing env-var switches; deploy the provided K8s manifests.
- **Feedback-driven improvement loop**: use stored like/dislike + groundedness data to auto-tune thresholds and routing heuristics.

32 Maintenance & Support

Area	Guidance
Configuration	All knobs live in backend/config/settings.py, documented in .env.example; never commit .env
Key modules to watch	services/chat_service.py (chat pipeline), retrieval/hybrid.py (quality), ingestion/service.py (jobs), security/* (guards)
Database maintenance	Run alembic upgrade head on deploy; monitor job_files table size; purge dead-letter jobs periodically

Area	Guidance
Vector store	Re-run warm_index after KB data changes; monitor collection size; enable quantization past ~5M vectors
Logging/monitoring	Set LOG_FORMAT=json in production; scrape /api/v1/metrics; alert on 5xx rate, retrieval p95, dead-letter count
Troubleshooting	X-Request-ID header correlates all log lines for a request; PipelineTrace in each chat done frame shows stage timings
Dependency upgrades	Pinned ranges in requirements.txt; run the offline pytest suite before bumps; CI verifies migrations
Special attention	Keep GEMINI_API_KEY rotated; extend injection patterns in security/injection.py as threats evolve

33 GitHub / Repository Readiness

Aspect	Assessment
README	Excellent: honest capability table, quick start, Docker, benchmarks, API surface, docs links
Docs	7 focused docs pages (architecture, rag-pipeline, security, scaling, evaluation, deployment, disaster-recovery) + CHANGELOG
.gitignore	Covers .venv, node_modules, model_cache, qdrant_storage, .env, caches
Secrets	None tracked; .env.example uses placeholders; compose reads env
Reproducibility	Pinned dependency ranges, frontend lockfile, seeded JSON KBs, benchmark scripts, generated dataset checked in
Tests / CI	66 offline tests + lint + migration smoke test + builds on every push/PR
Generated files	qdrant_storage/, db/chat.sqlite3, model_cache/ exist locally (gitignored); eval reports committed intentionally
Overall	High readiness — suitable for public portfolio with no sanitization work required

34 Resume / Portfolio Value

Skill	Evidence in the project
Full-stack development	React 19 + TypeScript SPA consuming a FastAPI streaming API end-to-end
Backend engineering	FastAPI, async SQLAlchemy 2.0, Alembic, lifespan management, middleware, DI
AI / RAG engineering	Gemini integration, ONNX embeddings, Qdrant, BM25+RRF+rerank, groundedness, evaluation harness
Database design	8-table relational schema with FKs, indexes, JSON columns, portable across PG/SQLite
Async & concurrency	asyncio pipelines, concurrent retrieval, threaded OCR, asyncio workers, backpressure
Security engineering	Rate limiting, tenant isolation, injection defenses, PII modes, upload validation, security headers
Observability	Prometheus metrics, request-ID correlation, structured logging, pipeline tracing
Testing & QA	66 offline pytest tests with fakes/ASGI transport; CI with lint + migration checks
DevOps	Multi-stage Docker, compose stack, GitHub Actions CI/CD, GHCR, Kubernetes manifests + HPA
Performance engineering	Multi-layer caching, latency/quality benchmarks with persisted reports
Technical writing	README honesty note, 7 docs pages, CHANGELOG, OpenAPI docs

35 Final Technical Summary

Dimension	Summary
Architecture	Layered async monorepo: React SPA → FastAPI → domain services → Qdrant/Redis/PostgreSQL, with dedicated workers; protocol adapters + fallbacks everywhere
Technology	Python 3.12 / FastAPI / SQLAlchemy 2 / Qdrant / Redis / Gemini 2.5 Flash; React 19 / TypeScript / Vite / Tailwind
Features	Agentic hybrid RAG with reranking, groundedness, memory, OCR+Vision, async ingestion, corpus management, live pipeline UI
Data flow	NDJSON streaming protocol with status/sources/tokens/groundedness/usage/trace; async job pipeline with audit trail
Security	Tenant isolation, optional API-key auth, rate limits, injection defense, PII modes, upload validation, secure headers; real auth is the top gap
Testing	66/66 offline pytest tests pass (executed); CI adds lint + migration checks; no frontend tests
Performance	Measured: hybrid retrieval p50 ~6 ms, rerank p50 ~33 ms, hit-rate 0.93 / MRR up to 1.0; ingestion ~1.5 docs/s (CPU ONNX)
Deployment	Docker + compose verified config; CI/CD to GHCR; K8s manifests + HPA ready but undeployed
Current status	Feature-complete locally (v2.0.0); productionization (auth, distributed deploy, object storage) remains

36 Appendix

36.1 Environment variables (names only — values are operator-provided)

GEMINI_API_KEY, GEMINI_MODEL, GEMINI_TIMEOUT_SECONDS, GEMINI_MAX_RETRIES, GEMINI_INPUT_PRICE_PER_MTOK, GEMINI_OUTPUT_PRICE_PER_MTOK, DATABASE_URL, DATABASE_POOL_SIZE, DATABASE_MAX_OVERFLOW, AUTO_CREATE_TABLES, REDIS_URL, QDRANT_URL, QDRANT_API_KEY, QDRANT_STORAGE_PATH, QDRANT_COLLECTION, QDRANT_HNSW_M, QDRANT_HNSW_EF_CONSTRUCT, QDRANT_QUANTIZATION, EMBEDDING_MODEL, EMBEDDING_BATCH_SIZE, RETRIEVAL_TOP_K, RETRIEVAL_CANDIDATES, RETRIEVAL_SCORE_THRESHOLD, MAX_CONTEXT_CHUNKS, RERANKING_ENABLED, RAG_STRATEGY, ADAPTIVE_RETRIEVAL_ENABLED, QUERY_ROUTER_MODE, MAX_CONTEXT_TOKENS, MEMORY_WINDOW_MESSAGES, GROUNDEDNESS_ENABLED, OCR_ENABLED, OCR_TIMEOUT_SECONDS, OCR_MAX_FILE_MB, OCR_LANGUAGE, OCR_VISION_MIN_CONFIDENCE, VISION_ENABLED, WORKER_CONCURRENCY, EMBEDDED_WORKER_ENABLED, MAX_RETRIES, MAX_QUEUE_DEPTH, INGESTION_MAX_FILE_MB, CACHE_TTL, RETRIEVAL_CACHE_TTL, EMBEDDING_CACHE_TTL, OCR_CACHE_TTL, ROUTER_CACHE_TTL, CHAT_RATE_LIMIT, OCR_RATE_LIMIT, INGESTION_RATE_LIMIT, PII_MODE, PROMPT_INJECTION_FILTER_ENABLED, API_KEY_AUTH_ENABLED, API_KEY, MAX_UPLOAD_MB, LOG_LEVEL, LOG_FORMAT, METRICS_ENABLED, CORS_ORIGINS, DEFAULT_TENANT_ID; frontend: VITE_API_BASE_URL.

36.2 Glossary

Term	Meaning
RAG	Retrieval-Augmented Generation — answering from retrieved corpus evidence
RRF	Reciprocal Rank Fusion — combines dense and BM25 ranked lists
Groundedness	LLM-as-judge verdict of how well the answer is supported by retrieved context
Dead-letter	Terminal state for jobs that exhausted retries

Term	Meaning
Tenant	Isolation key (X-Tenant-ID) scoping all data and retrieval
NDJSON	Newline-delimited JSON streaming protocol
HNSW	Hierarchical Navigable Small World — Qdrant's approximate-nearest-neighbor index

36.3 Verification environment

All verifications in Section 22 were executed in this workspace (Windows, Python 3.12 venv at backend/.venv, Node.js frontend toolchain). Test command: python -m pytest tests -q → 66 passed. Server verification used uvicorn main:app with default zero-config settings and no GEMINI_API_KEY. Benchmark numbers were taken from persisted reports in backend/eval/reports/. No API keys, passwords or tokens appear anywhere in this report.